REST Principles: Constraints, Resources & Statelessness

12 min read Updated 24 Jul 2026 Topic 39 of 70

Bookmark Mark done

Why it matters

Almost every backend job description asks you to "build and consume REST APIs," and almost every system-design interview eventually reaches the words "expose an endpoint." REST is the default vocabulary of the web: it is how browsers talk to servers, how mobile apps talk to backends, and how microservices talk to each other. If you understand REST deeply, you can design an interface that a stranger can use without reading your source code — and that is the entire point of an API.

But REST is also one of the most *misunderstood* topics in interviews. Most developers think "REST" means "JSON over HTTP with some URLs." That is not REST; that is just HTTP. REST is a specific **architectural style** defined by Roy Fielding in his year-2000 dissertation, and it is a set of **constraints** that, when you accept them, buy you real properties: scalability, cacheability, and the ability to evolve a server and its clients independently. Interviewers probe this gap constantly. The candidate who can name the constraints and explain *what each one buys you* stands out immediately from the one who just says "REST is stateless, I think."

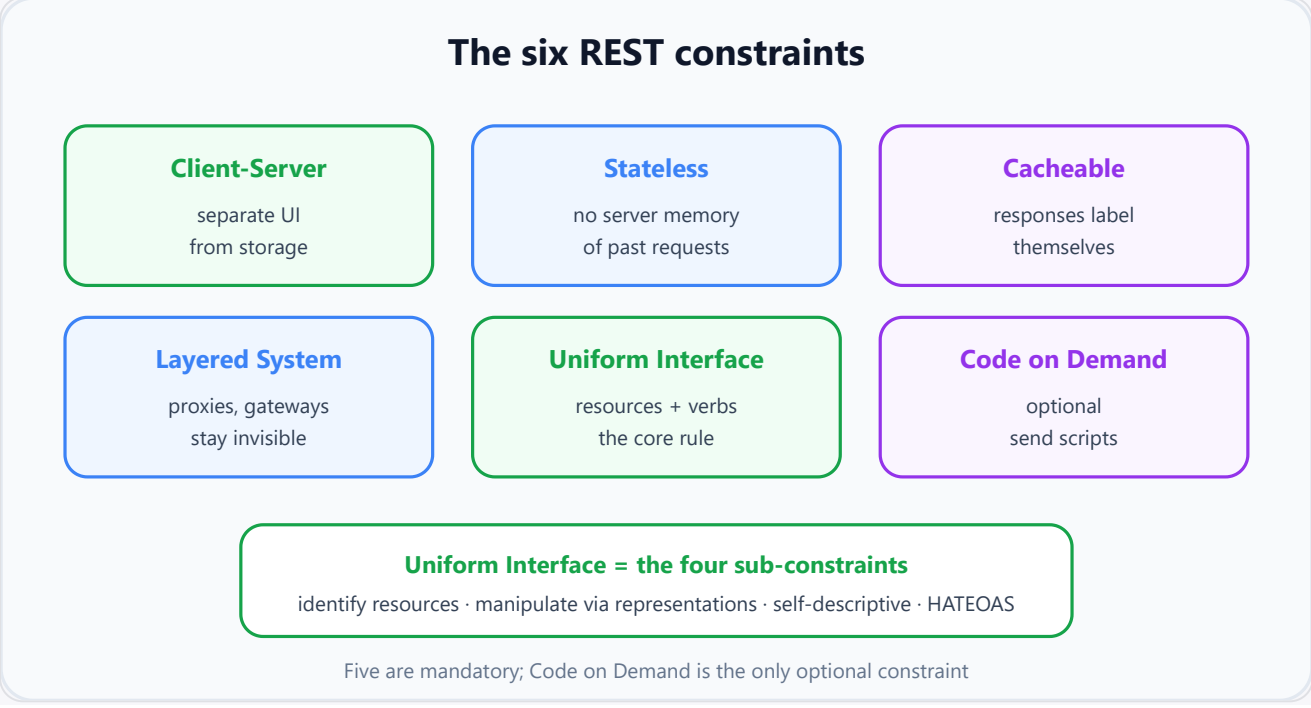
Getting REST right early also saves you from expensive rewrites. A stateful API that stores per-user data in server memory cannot be load-balanced across many machines. An API that ignores the uniform interface forces every client to be hand-coded against your quirks. These are not academic concerns — they are the difference between an API that scales to millions of users and one that falls over at ten thousand.

The concept, simply

REST stands for **REpresentational State Transfer**. Unpack the name and the whole idea falls out: a client transfers *representations* (usually JSON or HTML documents) of *resource state* back and forth with a server. A **resource** is any thing worth naming — a user, an order, a photo, a collection of comments. The resource itself lives on the server; what crosses the wire is a *representation* of it at a moment in time.

REST is defined by six constraints. Five are mandatory and one is optional. Accepting them is what makes an API "RESTful."

- **Client–Server:** separate the user interface from data storage. They evolve independently.
- **Stateless:** each request carries everything the server needs. The server keeps no memory of previous requests.
- **Cacheable:** responses must declare whether they can be cached, so clients and intermediaries can reuse them.
- **Uniform Interface:** the one big idea — resources are identified by URIs, manipulated through representations, messages are self-descriptive, and hypermedia drives state (HATEOAS).
- **Layered System:** a client cannot tell whether it is talking to the real server or a proxy, gateway, or cache in between.
- **Code on Demand (optional):** the server may ship executable code (e.g. JavaScript) to extend the client.



The six REST constraints. Five are mandatory; only Code on Demand is optional, and Uniform Interface is the one that most APIs get wrong.

The mental shortcut: **REST is about nouns, not verbs**. You do not have a `/getUser` function and a `/deleteUser` function. You have one noun — `/users/42` — and you use HTTP *verbs* (GET, DELETE) to act on it. The URL names the thing; the method names the action.

How it works (deep)

Let us take the constraints that carry the most interview weight and go deeper.

Statelessness is the constraint people quote but rarely explain correctly. It means the *server* stores no client *session context* between requests. Every request must contain all information needed to understand and process it — typically an auth token, not a server-side session pointer. This does *not* mean the application has no state; your database is full of state. It means the *conversation* is stateless. Why does this matter? Because if request N does not depend on the server remembering request N-1, then any of your ten identical servers can handle it. You can add servers behind a load balancer freely, restart one without losing sessions, and route each request anywhere. Statelessness is what makes horizontal scaling trivial.

The classic confusion: "But I use JWTs / cookies for auth — isn't that state?" No. The *token* is state that lives on the *client* and is sent with every request. The server validates it fresh each time and forgets it. Contrast that with an old-style server-side session where the cookie is just an ID pointing at memory *on the server* — that version is stateful and does not scale without sticky sessions or a shared session store.

The Uniform Interface is the constraint that actually makes REST feel like REST, and it has four sub-parts. (1) *Identification of resources*: each resource has a stable URI, e.g. `/orders/9`. (2) *Manipulation through representations*: the client never edits the resource directly; it sends a representation (a JSON body) describing the desired new state. (3) *Self-descriptive messages*: each message carries enough metadata — the method, `Content-Type`, status code — to be understood on its own. (4) *HATEOAS* (Hypermedia As The Engine Of Application State): responses include links telling the client what it can do next, so the client discovers actions rather than hard-coding URLs.

pay an order, POST to `/orders/9/pay`," the order response itself contains a link like `{ "rel": "pay", "href": "/orders/9/pay" }`. The client follows links the way a human follows hyperlinks on a web page — the server can move or rename endpoints and the client keeps working. In practice, very few JSON APIs are fully HATEOAS-driven (it adds complexity clients rarely exploit), but you should be able to explain it, because Fielding considered it the part *without which an API is not truly REST*. Be honest in interviews: "Full HATEOAS is rare in real JSON APIs; most teams ship 'REST-ish' APIs and that's a pragmatic, accepted trade-off."

Cacheability and the **layered system** work together. Because responses are self-descriptive and can be labeled cacheable (via `Cache-Control`, `ETag`, `Last-Modified`), a proxy sitting between client and server can serve a cached copy without the origin ever seeing the request. The client cannot tell — and does not need to — whether it hit the origin or a CDN edge. That invisibility is the layered-system constraint, and it is why CDNs and reverse proxies can accelerate a well-designed REST API for free.

Code examples

A minimal resource-oriented Express router. Notice the URL names a noun and the HTTP method carries the intent — there is no verb in the path.

JAVASCRIPT

Copy

```
const express = require('express');
const router = express.Router();

// Resource: the "users" collection and its members
router.get('/users', listUsers);         // read the collection
router.post('/users', createUser);       // create a member
router.get('/users/:id', getUser);       // read one member
router.put('/users/:id', replaceUser);   // replace one member
router.delete('/users/:id', deleteUser); // remove one member

module.exports = router;
```

Statelessness in practice: the server reads the identity from the token on *every* request and holds nothing between calls. No `req.session`, no in-memory user map.

Node.js Backend Kit

JAVASCRIPT Copy

```
const jwt = require('jsonwebtoken');

// Auth middleware — stateless: the token IS the context
function authenticate(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: 'Missing token' });

  try {
    // Verified fresh each request; server keeps no memory of it afterward
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ error: 'Invalid token' });
  }
}
```

A HATEOAS-flavored response — the resource tells the client what it can do next, instead of the client hard-coding paths.

JAVASCRIPT Copy

```
function getOrder(req, res) {
  const order = { id: 9, status: 'pending', total: 4200 };

  res.json({
    ...order,
    _links: {
      self:   { href: `/orders/${order.id}` },
      pay:    { href: `/orders/${order.id}/pay`,    method: 'POST' },
      cancel: { href: `/orders/${order.id}/cancel`, method: 'POST' },
    },
  });
}
```

And a self-descriptive, cacheable response using an `ETag` so a proxy or the browser can revalidate cheaply:

JAVASCRIPT Copy

```
function getProduct(req, res) {
  const product = { id: 7, name: 'Keyboard', version: 3 };
  const etag = `"prod-${product.id}-v${product.version}"`;

  res.set('Cache-Control', 'public, max-age=60');
  res.set('ETag', etag);

  if (req.headers['if-none-match'] === etag) {
    return res.status(304).end(); // Not Modified — client reuses its copy
  }
  res.json(product);
}
```

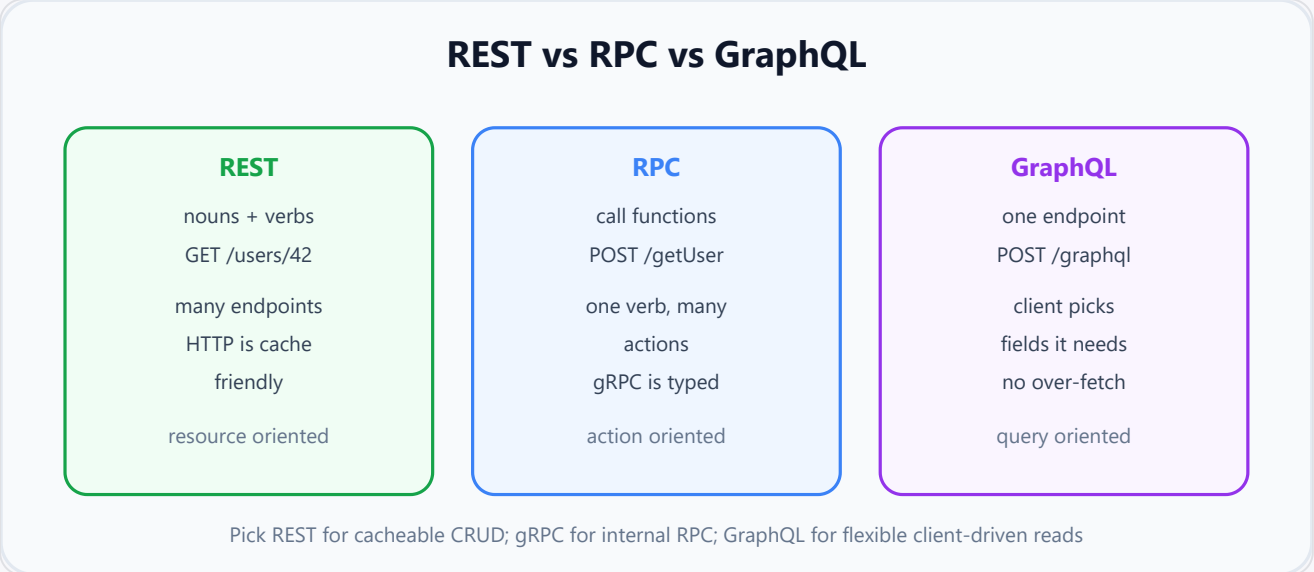
Common mistakes & gotchas

- **Putting verbs in URLs:** `/getUser`, `/createOrder`, `/api/deleteUser?id=3`. The method already carries the verb; the path is a noun. Verbs in paths are the #1 sign of an RPC-shaped API wearing a REST costume.
- **Confusing "stateless" with "no database."** Statelessness is about *per-client session* state on the server, not about persistence. Your DB is fine.
- **Server-side sessions in memory:** storing login state in a module-level object breaks the moment you run two instances — requests hit the wrong server. This is a statelessness violation, not a "bug."
- **Pluralization chaos:** mixing `/user/42` and `/orders/9`. Pick plural collection nouns and stay consistent.
- **Claiming full REST while hard-coding every URL client-side:** that is fine and common, but do not call it HATEOAS. Know the difference so you can answer honestly.
- **Nesting resources infinitely:** `/users/1/orders/9/items/3/reviews/2` is unusable. Nest one level for ownership, then link.

- Model **nouns as resources** and use HTTP methods as the verbs; keep paths free of action words.
- Use **plural collection names** (`/users`, `/orders`) consistently, and identify members by ID (`/users/42`).
- Keep the server **stateless**: carry auth in a token, and if you need shared session data, put it in Redis or the DB — never in process memory.
- **Declare cacheability** explicitly with `Cache-Control` and `ETag`; a self-descriptive response is a scalable one.
- **Nest shallowly.** One level of nesting to express ownership; beyond that, return links or use query filters.
- Be pragmatic about HATEOAS: add `_links` where they genuinely help clients discover actions, but do not over-engineer.
- Pick REST when your resources map cleanly to CRUD and you want HTTP caching; reach for gRPC or GraphQL when the fit is poor (see below).

REST vs RPC vs GraphQL

Interviewers often ask you to compare REST with its alternatives, because the comparison reveals whether you understand *why* REST is shaped the way it is.



REST names resources, RPC names actions, GraphQL lets the client shape the response. Each optimizes for a different pain.

procedure. It is great for internal service-to-service traffic where you control both ends, want a typed contract (Protobuf), and do not care about HTTP caching. **GraphQL** exposes one endpoint and lets the client ask for exactly the fields it needs, solving REST's over-fetching (getting more data than you use) and under-fetching (needing several round-trips). Its cost: HTTP caching and simple status-code semantics are harder because everything is a POST that returns 200. **REST** sits in the middle: resource-oriented, cache-friendly, universally tooled, and dead simple to consume with `curl` or a browser. A strong interview answer: "REST for public, cacheable, CRUD-shaped resources; gRPC for high-throughput internal RPC; GraphQL when many clients need flexible, differently-shaped reads from one graph."

⭐ Interview lens

Q: What does "stateless" really mean in REST, and why do we want it?
It means the server keeps no client session context between requests — every request carries everything needed to process it (typically an auth token). The trap is thinking it means "no database"; application state in the DB is fine. We want it because it enables horizontal scaling: any server can handle any request, so you can load-balance freely and restart instances without losing sessions.

Q: Name the REST constraints. Which is optional?
Client–Server, Stateless, Cacheable, Uniform Interface, Layered System, and Code on Demand. Code on Demand is the only optional one. Bonus points for naming the four sub-constraints of the Uniform Interface, especially HATEOAS.

Q: What is HATEOAS and do real APIs use it?
Hypermedia As The Engine Of Application State: responses include links telling the client what it can do next, so it discovers actions instead of hard-coding URLs. Honestly, full HATEOAS is rare in JSON APIs; most ship "REST-ish" APIs with hard-

coded routes. The trap is claiming your API is HATEOAS when it just returns data — interviewers respect the honest, nuanced answer.

Q: Is a JWT-based auth API stateful?

No. The token is state held by the *client* and sent each request; the server validates it fresh and forgets it. That is stateless. A server-side session where the cookie is an ID pointing at server memory *is* stateful — that's the distinction they're testing.

Q: How do you decide between REST and GraphQL?

REST when resources map cleanly to CRUD and you want HTTP caching and simple tooling. GraphQL when many diverse clients need to fetch differently-shaped data and REST forces over- or under-fetching. Trap: saying "GraphQL is just better" — it trades away easy caching and clear status semantics.

Q: Why is `POST /users/deleteAll` not RESTful?

It puts a verb in the URL and uses POST for a destructive bulk action instead of modeling resources. The RESTful shape is `DELETE` on specific resources; a bulk operation is better expressed as an operation resource or a filtered `DELETE`, not an RPC-style verb path.

Cheat-card

- **REST** = REpresentational State Transfer; an architectural *style*, not "JSON over HTTP."
- **6 constraints:** Client–Server, Stateless, Cacheable, Uniform Interface, Layered System, Code-on-Demand (optional).
- **Resources are nouns**; HTTP methods are the verbs. No verbs in paths.
- **Stateless** = no server-side session between requests ≠ no database.
- **Uniform Interface** sub-parts: identify resources, manipulate via representations, self-descriptive messages, HATEOAS.

- Use **plural collection names**, shallow nesting, and explicit cache headers.

QUICK CHECK — no marks, just for you.

1. In REST, what does "stateless" actually require of the server?

- ◯ It keeps no client session context between requests; each request carries everything needed to process it
- ◯ It stores no data at all, so a RESTful API cannot use a database
- ◯ It must keep every client session in server memory for fast lookup
- ◯ It caches every response so repeated requests never reach the origin

Check answer

2. Which of the six REST constraints is the only optional one?

- ◯ Uniform Interface
- ◯ Code on Demand
- ◯ Stateless
- ◯ Layered System

Check answer

3. Why is a JWT-based auth API still considered stateless?

- ◯ Because the server stores each issued token in memory and looks it up per request

Because JWTs are encrypted, so the server never has to read them

- ○ The token is state held by the client and sent each request; the server verifies it fresh and keeps no memory of it
- ○ Because tokens remove the need for any authentication on the server

Check answer

Previous
The req and res objects: reading and responding

Next
CRUD, HTTP Verbs, Status Codes & Idempotency

LEARN MORE

ARTICLES & DOCS

- REST - MDN Web Docs Glossary
  developer.mozilla.org
- An overview of HTTP
  developer.mozilla.org
- Web API Design Best Practices - Azure Architecture Center
  learn.microsoft.com

VIDEOS



- REST API concepts and examples
  WebConcepts



- APIs for Beginners - How to use an API (Full Course / Tutorial)
  freeCodeCamp.org

Hand-picked references for this lesson.

Node.js Backend Kit